

Software Architecture and Design Specification

Railway Ticket Booking System with Cancellation Feature

Document ID	SAD-RTBS-2026-V1	Date	15-09-2026
Version	1.0 (Final)	Status	Approved / Ready for Implementation
Authors	Software Architecture Team	Target System	Railway Ticket Booking System (RTBS)

Document Control

Revision History

Version	Date	Author	Change Summary
0.1	01-09-2026	Arch Team	Initial Draft & Outline
0.8	10-09-2026	Arch Team	Added Component Diagrams & Cancellation Sequence Workflows
1.0	15-09-2026	Lead Architect	Finalized STRIDE Threat Model, API Specs & Requirements Traceability

Approvals

Role	Name	Signature / Date
Chief Technology Officer	<Name>	15-09-2026
Lead Systems Architect	<Name>	15-09-2026
Head of Product & Operations	<Name>	15-09-2026

1. Introduction

1.1 Purpose

This document specifies the architectural structure, design patterns, component interactions, security model, and API contracts for the **Railway Ticket Booking System (RTBS)** with comprehensive cancellation and automated refund features. It serves as the primary technical baseline for developers, QA engineers, security auditors, and system administrators.

1.2 Scope

The RTBS architecture covers passenger registration, train schedule search, seat inventory lock/reservation, payment gateway integration, automated e-ticket issuance, multi-tier ticket cancellation (full, partial, waitlisted),

automated refund calculations based on dynamic cancellation policies, and external payment gateway reconciliations.

1.3 Audience

This document is intended for Software Engineers, Enterprise Architects, Quality Assurance Teams, DevOps Engineers, and Security Auditors overseeing the RTBS project.

1.4 Definitions & Acronyms

- **PNR:** Passenger Name Record - Unique 10-digit identifier for a booked itinerary.
- **RTBS:** Railway Ticket Booking System.
- **PCI-DSS:** Payment Card Industry Data Security Standard.
- **ADR:** Architectural Decision Record.
- **RAC:** Reservation Against Cancellation (Conditional seating allocation).
- **STRIDE:** Threat modeling framework (Spoofing, Tampering, Repudiation, Info Disclosure, DoS, Elevation of Privilege).

2. Document Overview

2.1 How to Use This Document

Readers should review Section 3 for macro-architectural design, component structures, and security considerations, followed by Section 4 for detailed low-level sequence flows, RESTful API specifications, and error-handling mechanics.

2.2 Related Documents

- RTBS Software Requirements Specification (SRS) v1.0
- RTBS Master Software Test Plan (STP) v1.0
- RTBS Security & PCI-DSS Compliance Guidelines v2.1

3. Architecture

3.1 Goals & Constraints

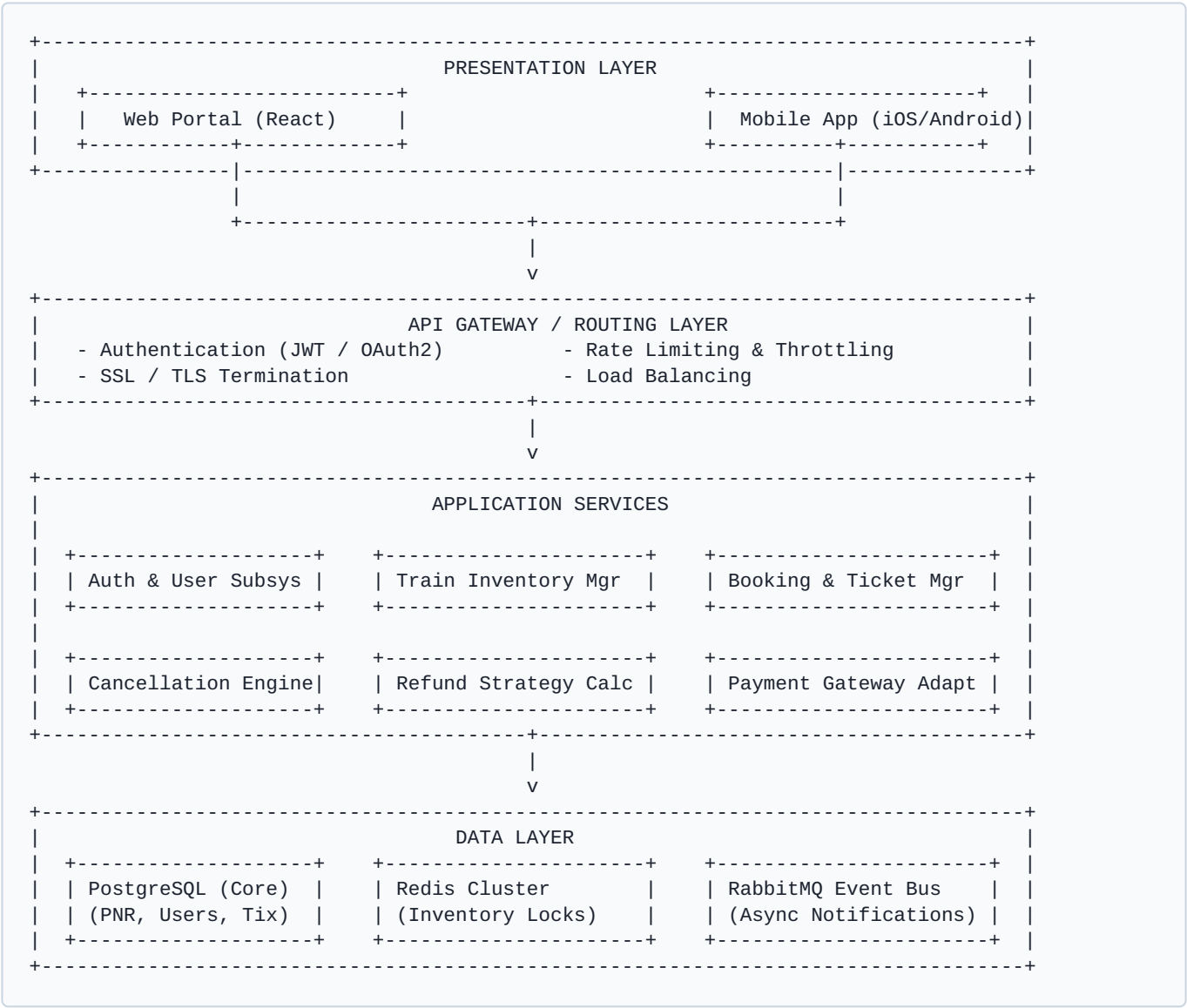
- **High Availability & Reliability:** 99.95% uptime with active-passive failover for high-volume booking hours.
- **Concurrency & Throughput:** Support up to 5,000 concurrent booking/cancellation transactions per second during peak hours.
- **Consistency:** Zero double-booking tolerance guaranteed via distributed transactional isolation and distributed locking (Redis/Redlock).
- **Regulatory Compliance:** Compliance with PCI-DSS v4.0 for payment tokenization and GDPR/Local Data Protection mandates.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Passengers	Instant seat booking, fast refund processing upon cancellation, transparent charge breakdown.
Railway Operations	Real-time inventory updating, accurate seat reuse after cancellation, anti-fraud controls.
Finance & Billing	Accurate refund deduction calculation, audit logs, automated bank gateway settlement.
DevOps / Security	System scalability, data encryption at rest and in transit, resilient service isolation.

3.3 Component Architecture (UML Class / Structure Diagram)

The system utilizes a modern Layered Services / Micro-component pattern. Below is the structural architectural component representation:



3.4 Component Descriptions

- **API Gateway:** Single entry point executing OAuth2 verification, dynamic rate limiting, and request routing.

- **Auth Service:** Manages passenger profiles, secure session tokens, password hashing (Argon2id), and RBAC.
- **Train Inventory Manager:** Controls real-time seat availability, route scheduling, and atomic seat locks using distributed Redis caching.
- **Booking & Ticket Engine:** Coordinates ticket generation, PNR assignment, passenger list binding, and status updates (Confirmed/RAC/Waitlisted).
- **Cancellation Engine:** Processes full/partial cancellation requests, releases reserved seats back to the inventory pool, and updates waitlisted passengers automatically.
- **Refund Strategy Calculator:** Evaluates cancellation rules against departure time windows, calculates net deductions, and computes net refund amount.
- **Payment Gateway Adapter:** Securely interfaces with third-party payment processors for charge processing and automated refund dispatch.

3.5 Chosen Architecture Pattern and Rationale

A **Layered Microservices / Event-Driven Hybrid Architecture** was chosen. Synchronous HTTP/REST flows are utilized for immediate booking and cancellation validation, while asynchronous AMQP messaging (RabbitMQ) is employed for refund dispatching, audit logging, and automated passenger notifications (SMS/Email). This prevents slow external payment gateways from blocking internal seat management operations.

3.6 Technology Stack & Data Stores

- **Programming Language / Framework:** Java 21 / Spring Boot 3.2.
- **Database:** PostgreSQL 16 (Relational consistency for PNR, transactions, and audit records).
- **Cache & Locking:** Redis 7 Cluster (Atomic seat locking via Redlock, high-speed schedule lookup).
- **Messaging:** RabbitMQ (Asynchronous event processing for notifications and refund execution).
- **Security:** Spring Security, JWT tokens, TLS 1.3 encryption in transit, AES-256 for data at rest.

3.7 Risks & Mitigations

Identified Risk	Impact	Architectural Mitigation
Race conditions during high-demand booking/cancellation	Double booking or double allocation of cancelled seats	Redis-based atomic distributed locks (Redlock) and database row-level locking (`SELECT ... FOR UPDATE`).
Payment Gateway timeout during refund dispatch	Unprocessed user refunds / financial discrepancies	Transactional outbox pattern with RabbitMQ retries and dead-letter queues (DLQ) for reconciliation.
Cascade service failure during peak rush	Complete system unavailability	Circuit Breakers (Resilience4j) at API Gateway level with fallback degraded modes.

3.8 Traceability to Requirements

Requirement ID	Requirement Summary	Mapped Component / Module
REQ-F-001	User Registration & Authentication	Auth & User Subsystem
REQ-F-002	Train Schedule & Availability Search	Train Inventory Manager (Redis Cache)

Requirement ID	Requirement Summary	Mapped Component / Module
REQ-F-003	Ticket Reservation & Payment Processing	Booking Engine & Payment Gateway Adapter
REQ-F-004	Ticket Cancellation (Full & Partial)	Cancellation Engine
REQ-F-005	Automated Refund Rule Calculation	Refund Strategy Calculator
REQ-NF-001	Response time < 2 seconds for search/cancellation	Redis Caching & Async RabbitMQ Processing

3.9 Security Architecture (STRIDE Threat Model)

Threat Category	Identified Threat	Mitigation Control
Spoofing	Unauthorized user attempting ticket cancellation for another passenger	JWT validation with cryptographically signed PNR ownership check before processing cancellation.
Tampering	Modification of refund amounts in HTTP payload	Server-side refund calculation inside Refund Calculator; client payloads only send PNR ID.
Repudiation	User claiming ticket was cancelled without consent	Immutable transaction and audit log stored in PostgreSQL with digital timestamp and IP tracking.
Info Disclosure	Interception of passenger PNR and credit card details	TLS 1.3 forced encryption for transport; PCI-DSS compliant tokenization for payment details.
Denial of Service	Bot attacks spamming cancellation API to deplete inventory	API Gateway rate limiting (10 req/min per IP/User) + CAPTCHA verification on cancellation.
Elevation of Privilege	Standard user attempting admin refund override	Strict Role-Based Access Control (RBAC) enforced by Spring Security filters.

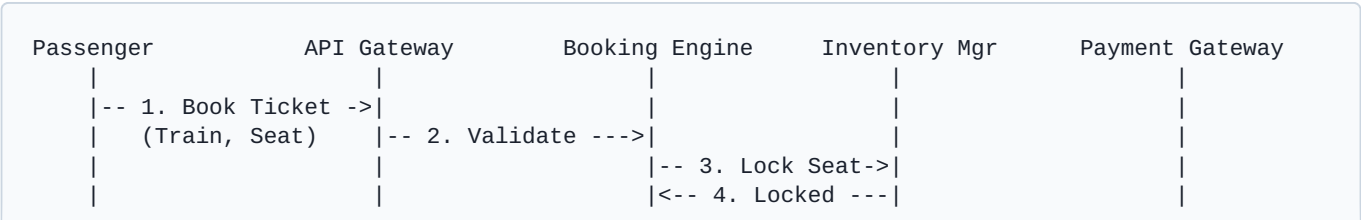
4. Design Specification

4.1 Design Overview

The design breaks down core business flows into distinct, modular workflows ensuring clean separation between presentation, business validation, inventory management, and financial processing.

4.2 Sequence Diagrams

Workflow 1: Ticket Booking Sequence Diagram



```
| | | | |
| | | |-- 5. Process Payment ----->|  
| | | |<-- 6. Payment Success -----|  
| | | |  
| | | |-- 7. Confirm Seat Allocation ----->|  
| | | |-- 8. Generate PNR & Ticket ----->|  
|<-- 9. Ticket Confirmed (PNR Issued) ---|
```

Workflow 2: Ticket Cancellation & Refund Sequence Diagram

```

Passenger      API Gateway      Cancellation Eng    Refund Calc        Inventory Mgr       Payment
|              |                  |                    |                   |                     |
|-- 1. Cancel Tix->|              |                    |                   |                     |
|   (PNR, TixID)  |-- 2. Validate ->|              |                   |                     |
|              |                  |-- 3. Get Rule -->|              |                     |
|              |                  |<--- 4. Net Refund-|              |                     |
|              |                  |              |              |                     |
|              |                  |-- 5. Release Seat Allocation --->|              |
|              |                  |   (Promote Waitlist Passenger)   |              |
|              |                  |              |              |                     |
|              |                  |-- 6. Initiate Refund Request ----->|              |
|              |                  |<--- 7. Refund Ack (Txn Ref ID) -----|              |
|<-- 8. Cancellation Confirmed & Refund Summary -|

```

4.3 API Design Specification

Endpoint 1: Create Ticket Cancellation Request

```
POST /api/v1/cancellation/process

Request Headers:
  Authorization: Bearer
  Content-Type: application/json

Request Body:
{
  "pnrNumber": "8429104821",
  "passengerIds": [101, 102],
  "cancellationReason": "Change of travel plans"
}

Response (200 OK):
{
  "cancellationId": "CAN-902184",
  "pnrNumber": "8429104821",
  "status": "SUCCESS",
  "cancelledSeats": ["B1-12", "B1-13"],
  "financialSummary": {
    "grossFare": 120.00,
    "cancellationFee": 20.00,
    "netRefundAmount": 100.00,
    "currency": "USD"
  },
  "refundReference": "REF-88410294",
  "timestamp": "2026-09-15T20:15:30Z"
}
```

```
Authorization: Bearer
Content-Type: application/json
```

```
{
  "pnrNumber": "8429104821",
  "passengerIds": [101, 102],
  "cancellationReason": "Change of travel plans"
}
```

```
{
  "cancellationId": "CAN-902184",
  "pnrNumber": "8429104821",
  "status": "SUCCESS",
  "cancelledSeats": ["B1-12", "B1-13"],
  "financialSummary": {
    "grossFare": 120.00,
    "cancellationFee": 20.00,
    "netRefundAmount": 100.00,
    "currency": "USD"
  },
  "refundReference": "REF-88410294",
  "timestamp": "2026-09-15T20:15:30Z"
```

```
}
```

Error Responses:

- 401 Unauthorized: Invalid or missing token.
- 404 Not Found: PNR does not exist.
- 400 Bad Request: Ticket already cancelled or train already departed.

Endpoint 2: Fetch Cancellation Fee & Refund Estimate

GET /api/v1/cancellation/estimate?pnrNumber=8429104821

Request Headers:

Authorization: Bearer

Response (200 OK):

```
{
  "pnrNumber": "8429104821",
  "departureTime": "2026-09-18T10:00:00Z",
  "currentTime": "2026-09-15T20:15:30Z",
  "hoursToDeparture": 61.75,
  "applicablePolicy": "FLAT_FEE_GREATER_48_HRS",
  "grossFare": 120.00,
  "estimatedDeduction": 20.00,
  "estimatedRefund": 100.00
}
```

4.4 Error Handling, Logging & Monitoring

- **Standardized Error Format:** RFC 7807 Problem Details for HTTP APIs.
- **Logging:** Structured JSON logging via Logback sent to ELK Stack (Elasticsearch, Logstash, Kibana). Strict PCI-DSS sanitization ensures PII and payment card numbers are masked.
- **Monitoring & Alerting:** Prometheus metrics exposed at `/actuator/prometheus` monitored by Grafana dashboards. Alerts trigger on cancellation failure rates exceeding 1.5%.

4.5 UX & Accessibility Design

- Clear visualization of cancellation policies prior to confirmation.
- Two-step confirmation modal showing exact breakdown of ticket fare, deductions, and final refund credit amount.
- WCAG 2.1 AA compliance for high contrast readability and screen-reader accessibility across Web and Mobile UI.

4.6 Open Issues & Next Steps

- **Issue 1:** Automatic instant-refund processing during bank gateway maintenance windows.
- **Next Step:** Implement a secondary fallback payment aggregator and dynamic refund queueing mechanism for offline processing.

5. Appendices

5.1 Glossary

PNR: Passenger Name Record. **Redlock:** Distributed lock algorithm using Redis. **RAC:** Reservation Against Cancellation.

5.2 References

IEEE 42010 Architecture Standard, PCI-DSS Security Guidelines v4.0, Spring Boot & Microservices Design Patterns.